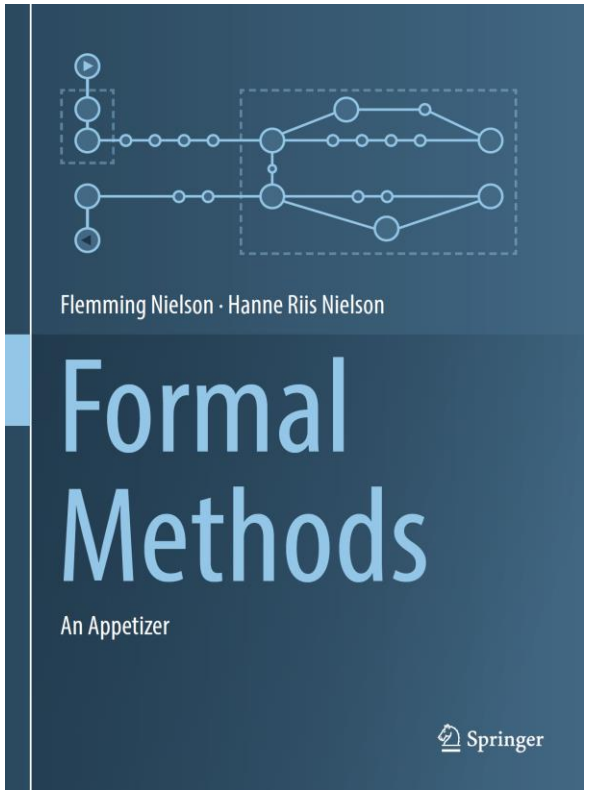
02141 – Computer Science Modelling

Formal Methods 01

Alberto Lluch Lafuente

(based on slides by Christoph Matheja)

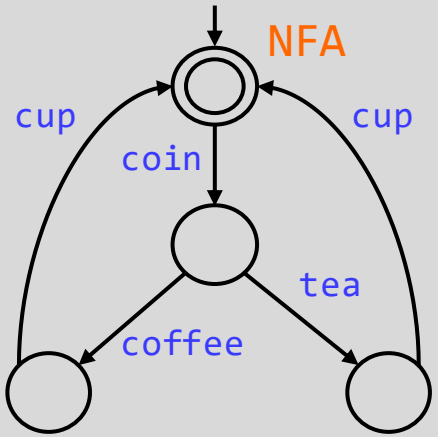


What are formal methods?

- Models describe real-world behavior
  - focus on some parts
  - abstract from other parts
- Our focus: computational models
  - automata, programs, Turing machines...
- Formal models have a precise semantics
  - i.e. their meaning is rooted in mathematics
- Formal methods are techniques for reasoning about formal models

R: (coin (coffee+tea) cup)*

$$L(\mathrm{R}) = \{(\mathrm{coin\ coffee\ cup})^n \mid n \in \mathbb{N}\} \cup \{(\mathrm{coin\ tea\ cup})^n \mid n \in \mathbb{N}\}$$



L(R) = L(NFA)?

L(NFA) = ∅?

Formal Models in Action

https://dl.acm.org/doi/pdf/10.1145/3282510

DOI:10.1145/3282510

Bringing the Web Up to Speed with WebAssembly

By Andreas Rossberg, Ben L. Titzer, Andreas Haas, Derek L. Schuff, Dan Gohman, Luke Wagner, Alon Zakai, J.F. Bastien, and Michael Holman

abstraction over modern hardware, making it independent of language, hardware, and platform and applicable far beyond just the Web. WebAssembly is the first mainstream language that has been designed with a formal semantics from the start, finally utilizing formal methods that have matured in programming language research over the last four decades.

1. INTRODUCTION

The Web began as a simple hypertext document network

is ty
eith
pile
runt
a ste
U
gran
or p
war
bee
min

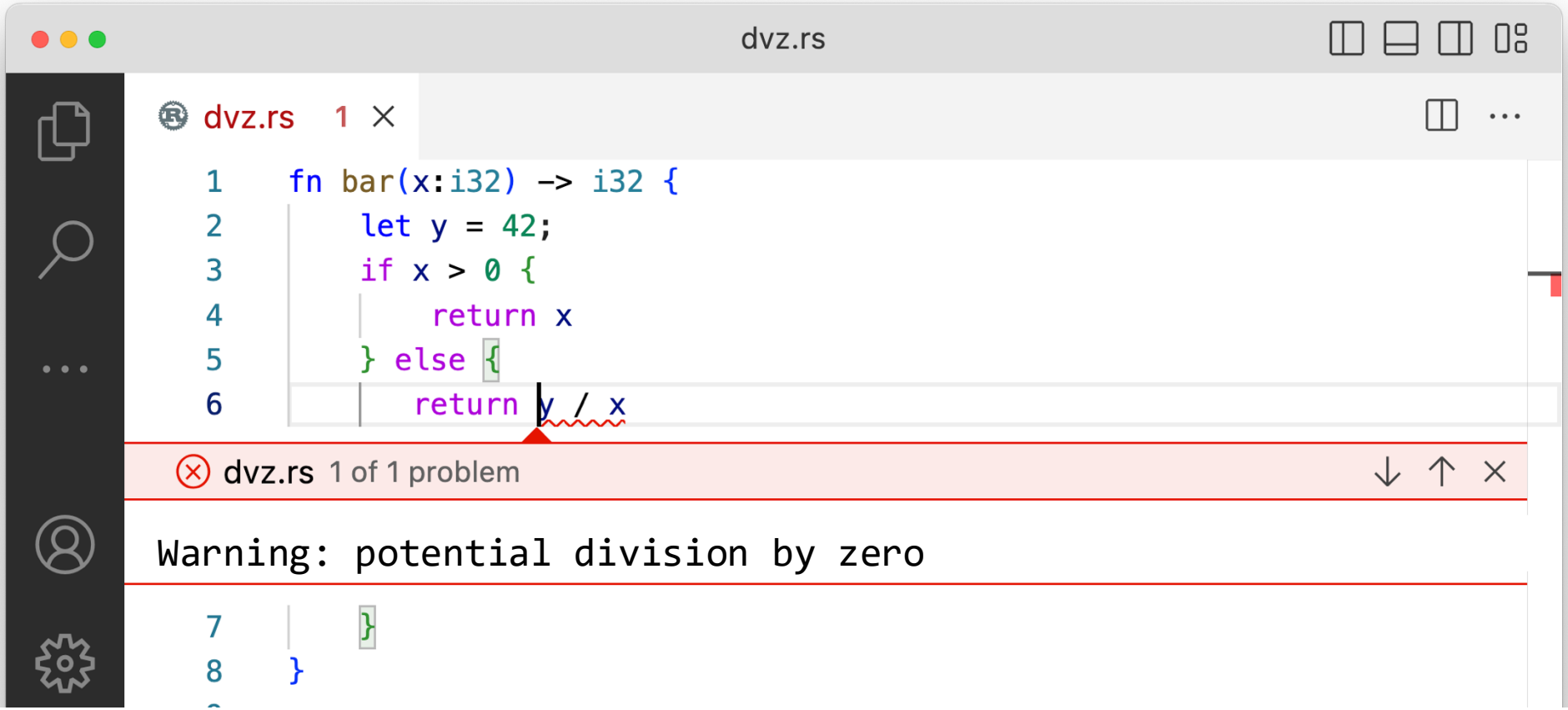
Formal Methods in Action

"The world's most highly assured operating system kernel."

What does seL4's formal verification mean?

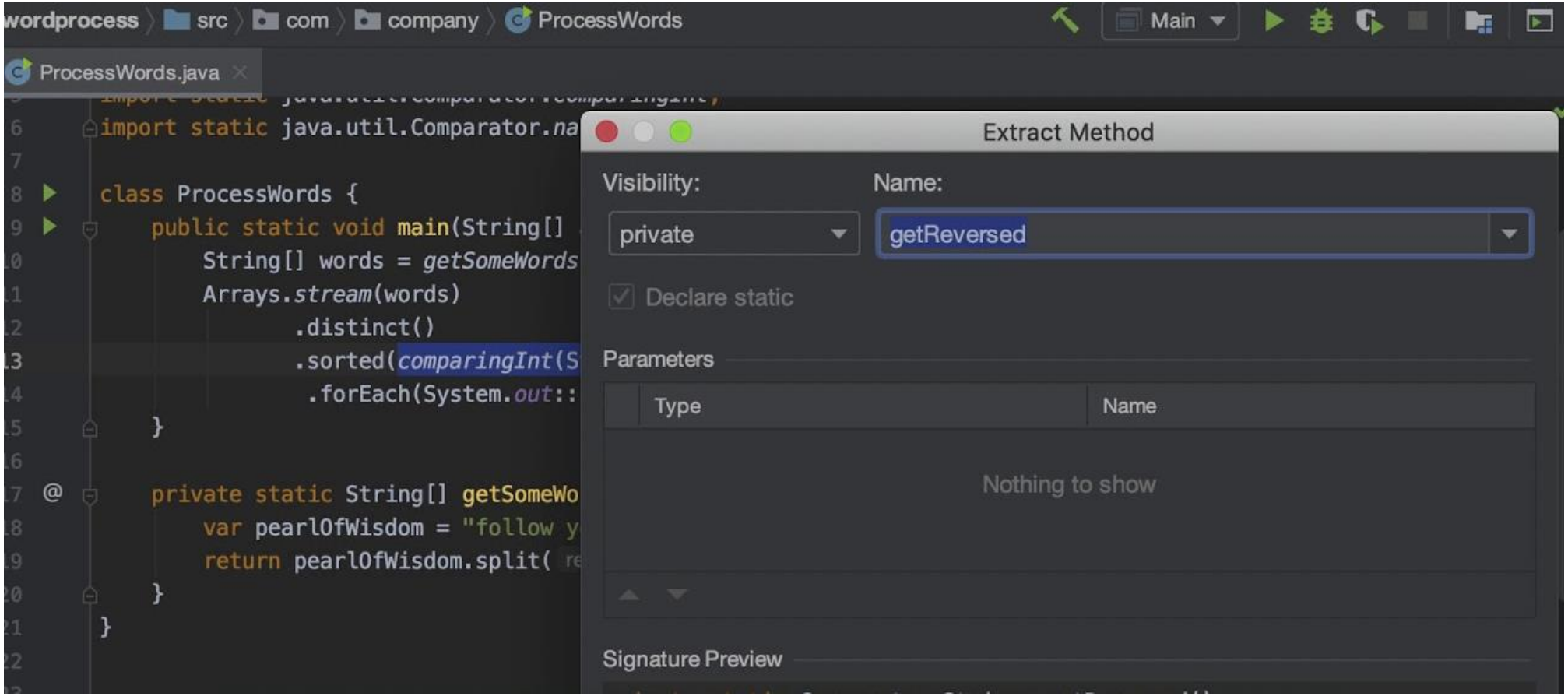
Unique about seL4 is its unprecedented degree of assurance, achieved through formal verification. Specifically, the ARM, ARM_HYP (ARM with virtualisation extensions), X64, and RISCV64 versions of seL4 comprise the first (and still only) general-purpose OS kernel with a full code-level functional correctness proof, meaning a mathematical proof that the implementation (written in C) adheres to its specification. In short, the implementation is proved to be bug-free (see below). This also implies a number of other properties, such as freedom from buffer overflows, null pointer exceptions, use-after-free, etc.

Formal Methods in Action



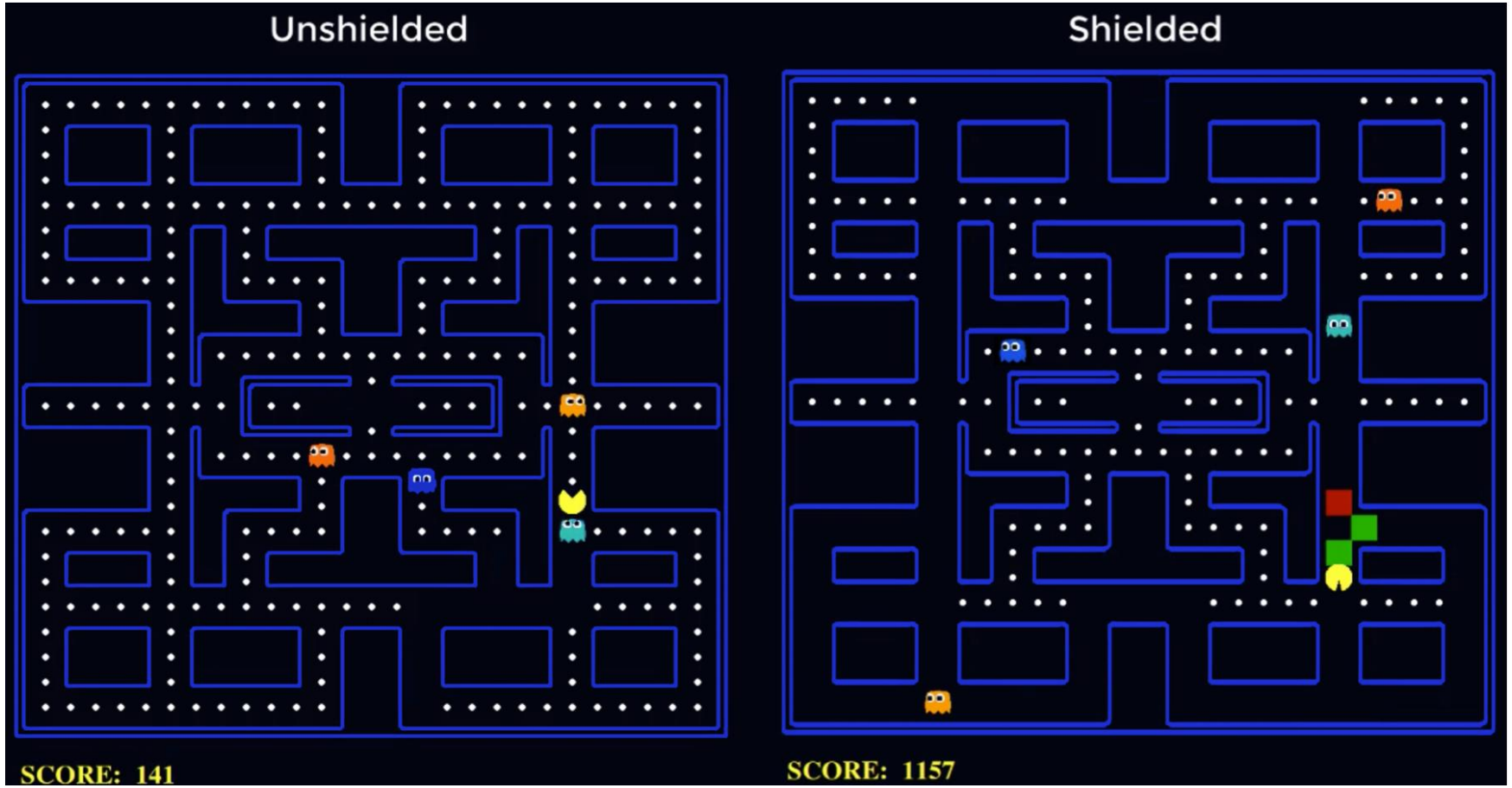
Static program analyses detect potential runtime errors at compile time

Formal Methods in Action



Refactoring tools need to preserve the meaning of the original program

Formal Methods in Action



A monitor based on a formal model prevents players from moves that would lose the game.

Formal Methods in Action – Wrap-Up

- Formal methods are at the foundation of software engineering

- Basis of many useful tools
  - Compilers, Interpreters
  - Integrated Development Environments
  - Program Analyses, Verifiers,
  - Monitors, Debuggers

- Formal methods must be rooted in models that capture the semantics of the systems they describe

Computer Scientists Close In on Perfect, Hack-Proof Code

Boya Sun for Quanta Magazine

The importance of *precise* semantics

- What does the following program compute?

```
int abs(int x)
{
  if (x >= 0) {
    return x;
  } else {
    return (-1) * x;
  }
}
```

The importance of *precise* semantics

- What does the following program compute?

```
int abs(int x)
{
  if (x >= 0) {
    return x;
  } else {
    return (-1) * x;
  }
}
```

- The absolute value of x?
  - Yes, if **int** represents all integers
  - No, if **int** represents 32-bit integers represented in two's complement

```
INT_MIN     is     -2_147_483_648i32
INT_MAX     is      2_147_483_647i32

abs(INT_MIN) is undefined!
```

Agenda for Today

1. Program Graphs – a formal model for control flow

2. Operational Semantics – a formal model for program behavior

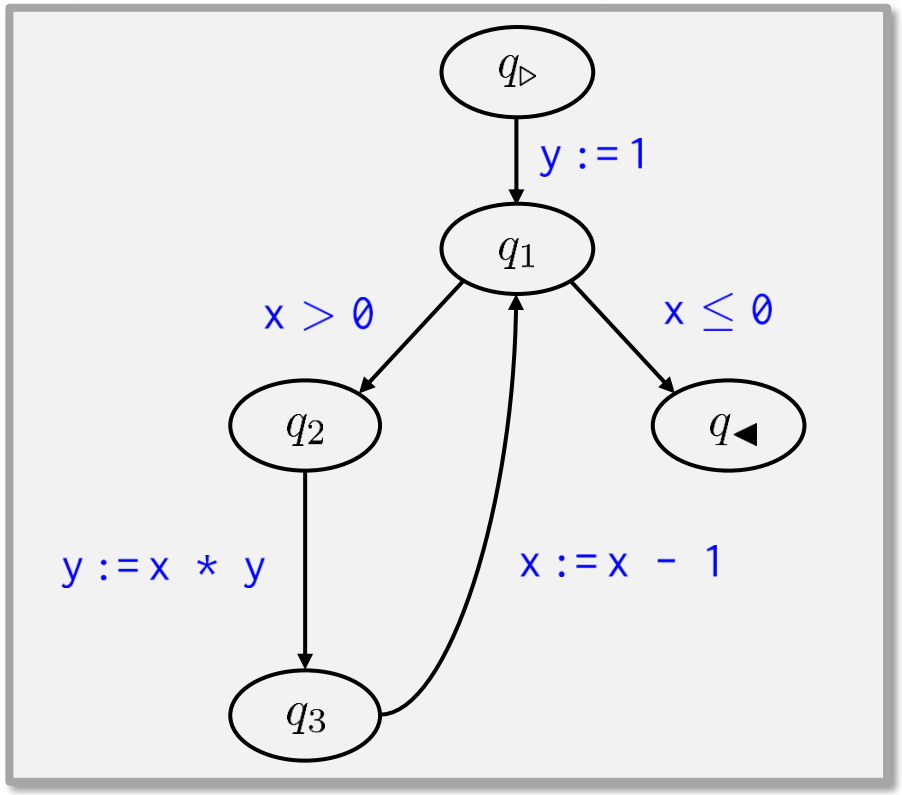
3. Properties of Program Graphs – two simple formal methods

02141 – Computer Science Modelling

PROGRAM GRAPHS

Program Graphs

```
y := 1;
while (x > 0) {
  y := x * y; x := x - 1
}
```



Definition. A **program graph** is a tuple

$\mathbf{PG} = (\mathbf{Q}, q_{\triangleright}, q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$, where

- $\mathbf{Q}$ is a finite set of nodes,

$$\mathbf{Q} = \{ q_{\triangleright}, q_1, q_2, q_3, q_{\blacktriangleleft} \}$$

- $q_{\triangleright} \in \mathbf{Q}$ is the initial node,
- $q_{\blacktriangleleft} \in \mathbf{Q}$ is the final node,

- $\mathbf{Act}$ is a set of actions,

$$\mathbf{Act} = \{ \texttt{y := 1}, \texttt{x} > \texttt{0}, \texttt{x} \leq \texttt{0}, \texttt{y := x * y}, \texttt{x := x - 1} \}$$

- $\mathbf{E} \subseteq \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$ is a finite set of edges

$$\begin{aligned}\mathbf{E} = \{&(q_{\triangleright}, \texttt{y := 1}, q_1), (q_1, \texttt{x} > \texttt{0}, q_2),\\ &(q_2, \texttt{y := x * y}, q_3), (q_3, \texttt{x := x - 1}, q_1),\\ &(q_1, \texttt{x} \leq \texttt{0}, q_{\blacktriangleleft})\}\end{aligned}$$

Construct program graphs for the following programs

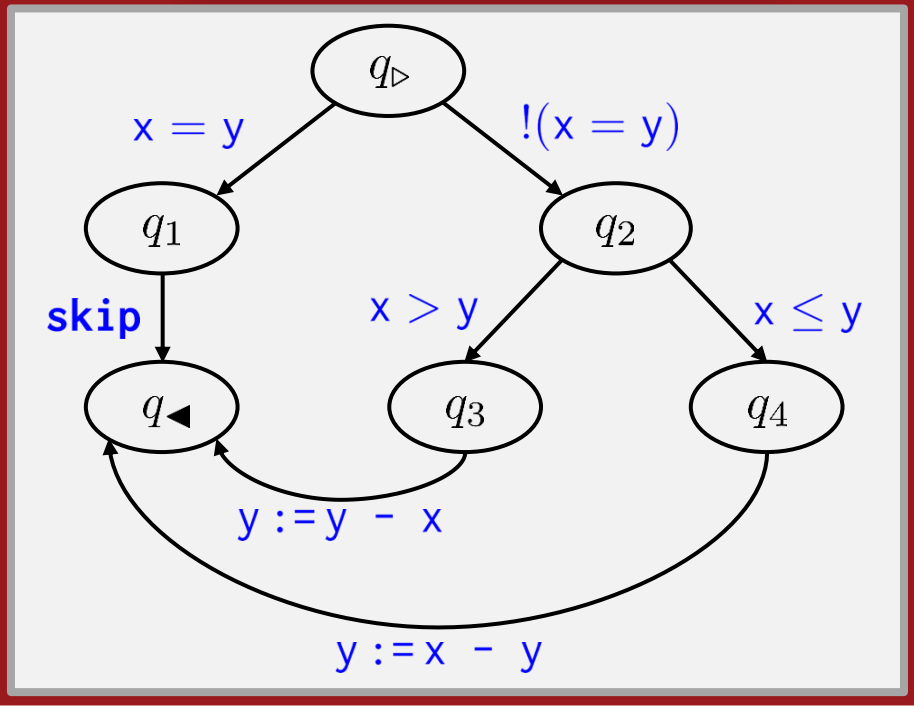
```
if (x == y) {
  skip
} else if (x > y) {
  y := y - x
} else {
  y := x - y
}
```

```
y := 0;
do {
  y := y + x * x;
  x := x - 1
} while (x >= 0)
```

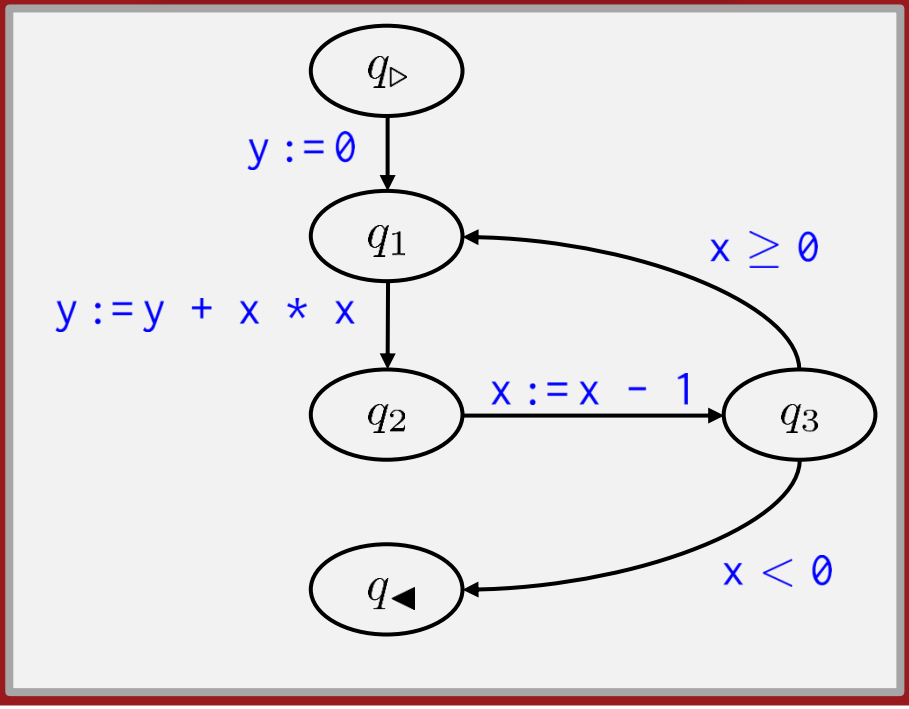
Actions are marked in blue (+ negations)

Solution: Construct program graphs for the following programs

```
if (x == y) {
  skip
} else if (x > y) {
  y := y - x
} else {
  y := x - y
}
```




```
y := 0;
do {
  y := y + x * x;
  x := x - 1
} while (x >= 0)
```



Program Graphs as Formal Models

- Focuses on possible sequences of actions
  - Assignments `x := y + 1`, Boolean tests `x * x > y`
- Abstracts from control flow statements and data stored in memory
  - `while, repeat-until, goto, if-then-else, try-catch, break, continue, ...`
- Semantics: what is the *meaning* of a program graph?
  - Abstract semantics: possible sequences of actions of the modelled program
  - Operational semantics: sequences of memory changes of actual program executions
- Enables formal analysis of program properties
  - Does a program always stop? Does it stop with x > y? Can we safely remove variable y?
  - Is x ever smaller than y? Does any variable ever become negative?

02141 – Computer Science Modelling

OPERATIONAL SEMANTICS

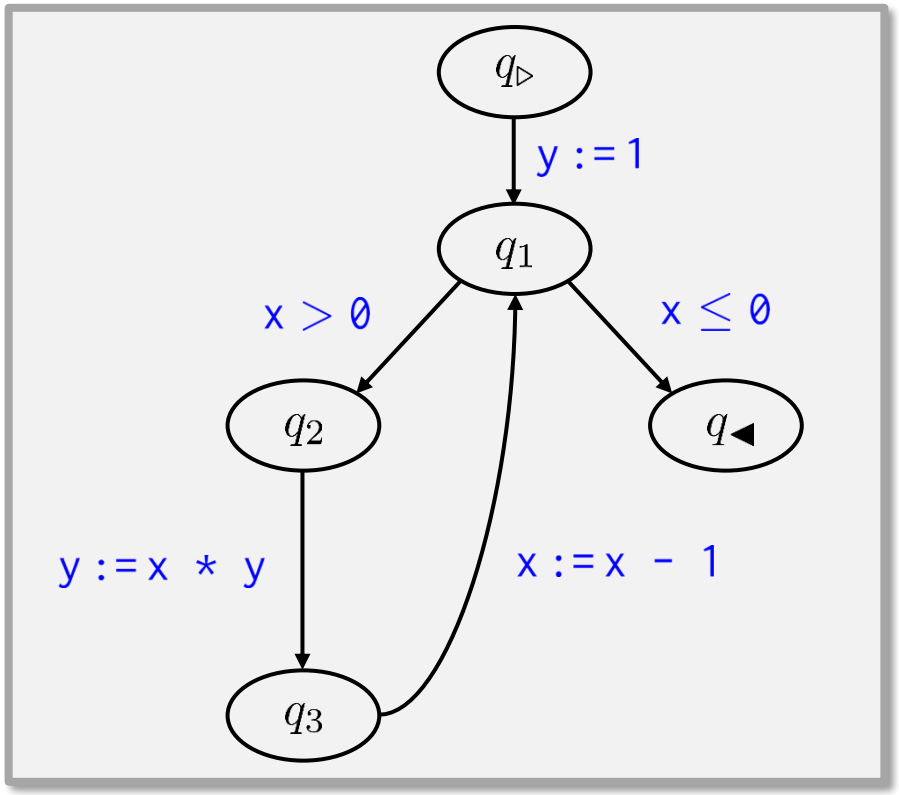
Operational Semantics

- **Goal:** *formalize* how programs are executed step by step
  - **Component 1:** control-flow (program counter) modeled by a program graph
  - **Component 2:** semantics of actions modeling how a program interacts with memory

- Mathematical foundations for reasoning about program behavior
  - Does a program always stop?
  - Has this program been correctly compiled?
  - Does a program always return the same output for a given input?
  - Can we safely remove variable y?
  - Is x always smaller than y?
  - Does any variable ever become negative?

Operational Semantics

- **Goal:** formalize how programs are executed step by step
  - **Component 1:** control-flow (program counter) modeled by a program graph
  - **Component 2:** semantics of actions modeling how a program interacts with memory



$$
\begin{aligned}
& (q_{\triangleright}; [\mathsf{x} = 2, \mathsf{y} = 0]) \\
\overset{\mathsf{y := 1}}{\Rightarrow}\ & (q_1; [\mathsf{x} = 2, \mathsf{y} = 1]) \\
\overset{\mathsf{x > 0}}{\Rightarrow}\ & (q_2; [\mathsf{x} = 2, \mathsf{y} = 1]) \\
\overset{\mathsf{y := x * y}}{\Rightarrow}\ & (q_3; [\mathsf{x} = 2, \mathsf{y} = 2]) \\
\overset{\mathsf{x := x - 1}}{\Rightarrow}\ & (q_1; [\mathsf{x} = 1, \mathsf{y} = 2]) \\
\overset{\mathsf{x > 0}}{\Rightarrow}\ & (q_2; [\mathsf{x} = 1, \mathsf{y} = 2]) \\
\overset{\mathsf{y := x * y}}{\Rightarrow}\ & (q_3; [\mathsf{x} = 1, \mathsf{y} = 2]) \\
\overset{\mathsf{x := x - 1}}{\Rightarrow}\ & (q_1; [\mathsf{x} = 0, \mathsf{y} = 2]) \\
\overset{\mathsf{x \leq 0}}{\Rightarrow}\ & (q_{\blacktriangleleft}; [\mathsf{x} = 0, \mathsf{y} = 2])
\end{aligned}
$$

"initial memory"

- value assigned to each variable

$\Rightarrow$: "execution step"

- update program counter
- update program memory based on semantics of performing an action

"final memory"

Memory (or *program state*)

- Assumptions:
  - **Var** is the (finite) set of program variables
  - All variables are of type unbounded integer

Definition. A **memory** is a function

$$\sigma : \mathbf{Var} \to \mathbf{Int}$$

that assigns an (integer) value to every variable.

v	σ(v)
x	12
y	5

memory σ

v	σ[y ⟼ 7](v)
x	12
y	7

updated memory σ[y ⟼ 7]

- **Mem** is the set of all memories
- The value of variable x is given by $\sigma(\mathrm{x})$
- $\sigma[\mathrm{x} \mapsto \mathtt{i}]$ is the same function as σ except that value $\mathtt{i}$ is assigned to x

Definition (memory update).

$$\sigma[\mathrm{x} \mapsto \mathtt{i}](\mathrm{y}) = \begin{cases} \mathtt{i}, & \text{if } \mathrm{y} = \mathrm{x} \\ \sigma(\mathrm{y}), & \text{if } \mathrm{y} \neq \mathrm{x} \end{cases}$$

Semantics of actions (idea)

To define the meaning of an action α, we describe how α transforms the input memory σ to an output memory.

➔ α determines a function **Mem** → **Mem**

Actually: for some input memories the output memory may be undefined

➔ α determines a partial function

Mem ↪ **Mem**

v	σ(v)
x	0
y	5

- Given: memory σ

- What is the effect of performing action `y := 7` on memory σ?

➔ σ changes to σ[y ⟼ 7]

- What is the effect of performing action `x >= 0` on memory σ?

➔ σ remains unchanged

(but this might affect the control flow)

- What is the effect of performing action `y := y / x` on memory σ?

➔ *undefined* (no follow-up memory)

Semantics of actions (formally)

Definition. A **semantic function**

$$\mathcal{S}[\![\cdot]\!] : \textbf{Act} \to (\textbf{Mem} \hookrightarrow \textbf{Mem})$$

maps every action $\alpha \in \textbf{Act}$ to a *partial* function

$$\textbf{Mem} \hookrightarrow \textbf{Mem}$$

specifying the effect of performing action α on any given memory.

$\mathcal{S}[\![\alpha]\!](\sigma)$ is the output memory σ' obtained from performing action α on input memory σ.

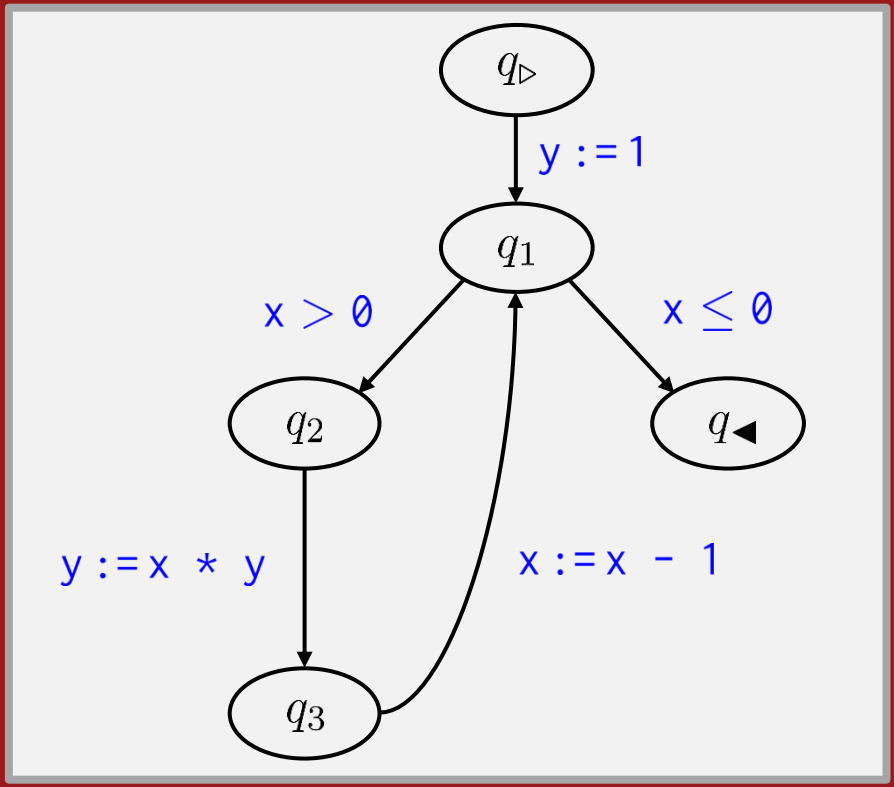
$\mathcal{S}[\![\alpha]\!](\sigma) =$ undefined if no suitable output memory exists for input memory σ.

$$\mathcal{S}[\![\texttt{y := 1}]\!](\sigma) = \sigma[\mathsf{y} \mapsto 1]$$

$$\mathcal{S}[\![\texttt{x} > \texttt{0}]\!](\sigma) = \begin{cases} \sigma, & \text{if } \sigma(\mathsf{x}) > 0 \\ \text{undefined}, & \text{if } \sigma(\mathsf{x}) \leq 0 \end{cases}$$

Exercise

- Define the semantic function $\mathcal{S}[\![\,\alpha\,]\!](\sigma)$ for all actions α in the program graph



$$\mathcal{S}[\![\,\texttt{y := 1}\,]\!](\sigma) \;=\; \sigma[\mathsf{y} \mapsto 1]$$

$$\mathcal{S}[\![\,\texttt{x > 0}\,]\!](\sigma) \;=\; \begin{cases} \sigma, & \text{if } \sigma(\mathsf{x}) > 0 \\ \text{undefined}, & \text{if } \sigma(\mathsf{x}) \leq 0 \end{cases}$$

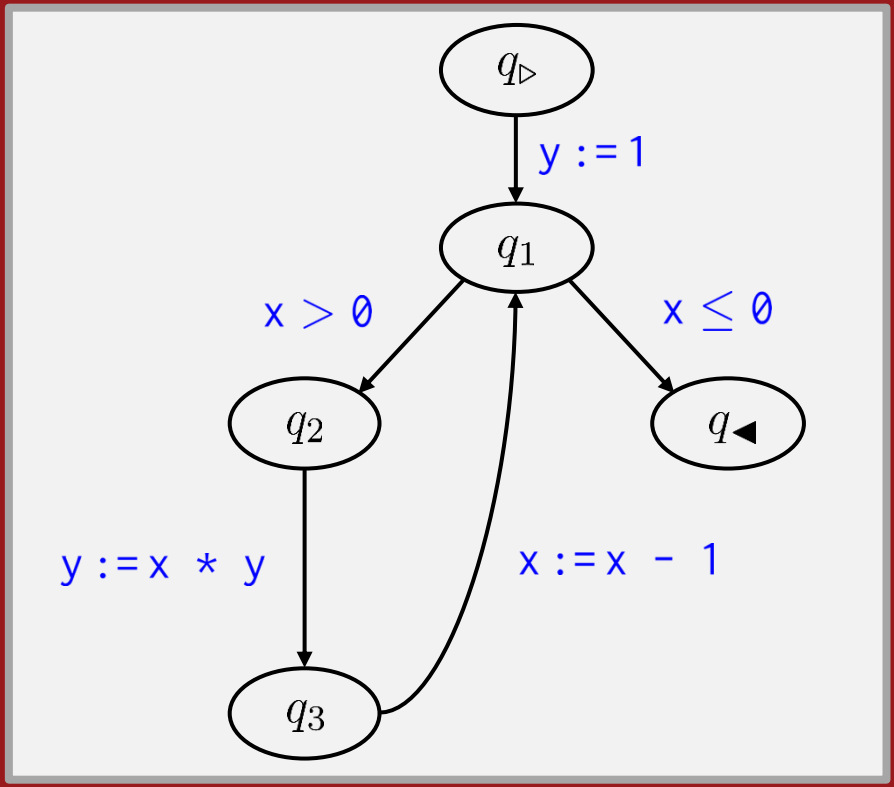
$$\mathcal{S}[\![\,\texttt{x} \leq \texttt{0}\,]\!](\sigma) \;=$$

$$\mathcal{S}[\![\,\texttt{x := x - 1}\,]\!](\sigma) \;=$$

$$\mathcal{S}[\![\,\texttt{y := x * y}\,]\!](\sigma) \;=$$

Solution

- Define the semantic function $\mathcal{S}[\![\,\alpha\,]\!](\sigma)$ for all actions α in the program graph



$$\mathcal{S}[\![\,\texttt{y := 1}\,]\!](\sigma) \;=\; \sigma[\mathsf{y} \mapsto 1]$$

$$\mathcal{S}[\![\,\texttt{x > 0}\,]\!](\sigma) \;=\; \begin{cases} \sigma, & \text{if } \sigma(\mathsf{x}) > 0 \\ \text{undefined}, & \text{if } \sigma(\mathsf{x}) \leq 0 \end{cases}$$

$$\mathcal{S}[\![\,\texttt{x} \leq \texttt{0}\,]\!](\sigma) \;=\; \begin{cases} \sigma, & \text{if } \sigma(\mathsf{x}) \leq 0 \\ \text{undefined}, & \text{if } \sigma(\mathsf{x}) > 0 \end{cases}$$

$$\mathcal{S}[\![\,\texttt{x := x - 1}\,]\!](\sigma) \;=\; \sigma[\mathsf{x} \mapsto \sigma(\mathsf{x}) - 1]$$

$$\mathcal{S}[\![\,\texttt{y := x * y}\,]\!](\sigma) \;=\; \sigma[\mathsf{y} \mapsto \sigma(\mathsf{x}) * \sigma(\mathsf{y})]$$

Configurations

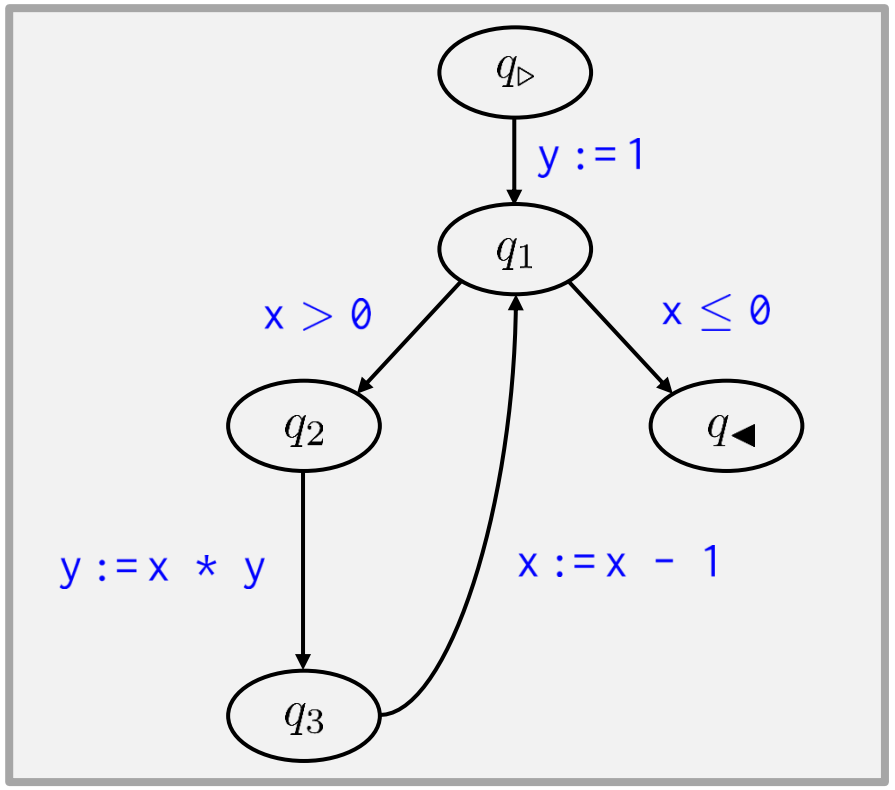
Definition. A **configuration** is a pair $\langle q; \sigma \rangle$ consisting of

1. a node $q \in \mathbf{Q}$ *that models the program counter, i.e. which actions can be performed next* and
2. a memory $\sigma \in \mathbf{Mem}$ *that models the data accessible by the program*.

Definition. A configuration $\langle q; \sigma \rangle$ is

- an **initial configuration** if $q = q_{\triangleright}$ and
- a **final configuration** if $q = q_{\blacktriangleleft}$.

We often write $\langle q_{\triangleright}; (3, 7) \rangle$ instead of $\langle q_{\triangleright}; \sigma[\mathsf{x} \mapsto 3][\mathsf{y} \mapsto 7] \rangle$ if the set of variables and their ordering is clear.



Example configurations:

$\langle q_{\triangleright}; \sigma[\mathsf{x} \mapsto 3][\mathsf{y} \mapsto 7] \rangle$

$\langle q_1; \sigma[\mathsf{x} \mapsto 3][\mathsf{y} \mapsto 1] \rangle$

$\langle q_{\blacktriangleleft}; \sigma[\mathsf{x} \mapsto 0][\mathsf{y} \mapsto 6] \rangle$

Execution Steps

We run program graphs by performing execution steps

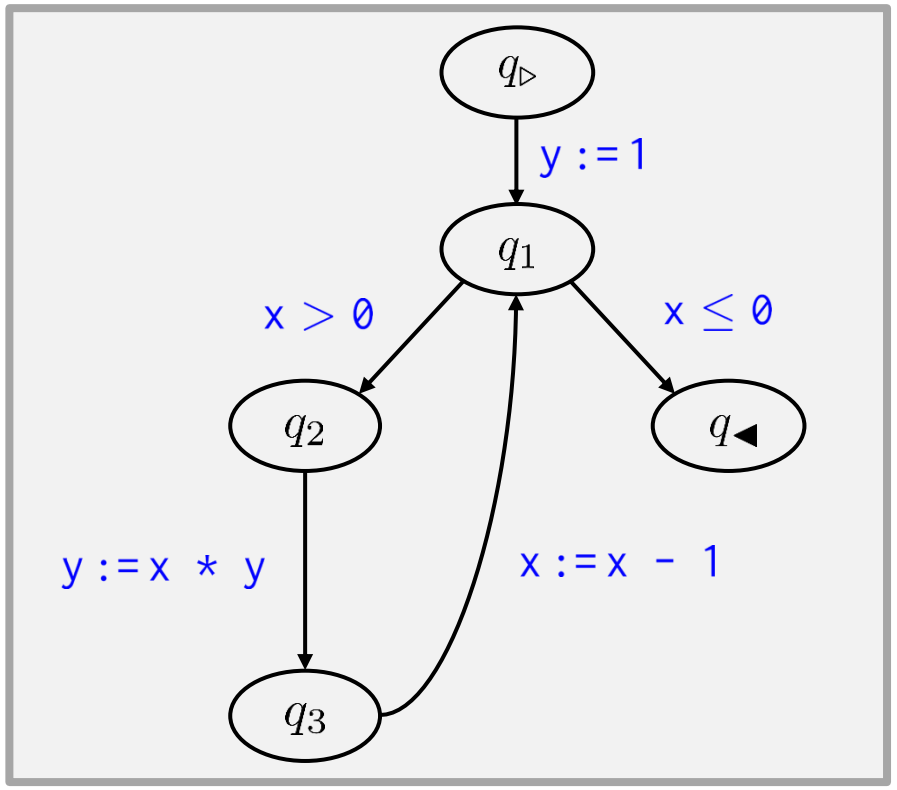
- Update program counter using edge relation **E**
- Update memory using semantic function $\mathcal{S}[\![\cdot]\!]$
- Updating memory undefined ➔ no execution step possible

Definition. We can perform an **execution step**

$$\langle q_\circ ; \sigma \rangle \overset{\alpha}{\Longrightarrow} \langle q_\bullet ; \sigma' \rangle$$

whenever

- $(q_\circ, \alpha, q_\bullet) \in \mathbf{E}$ and
- $\mathcal{S}[\![\alpha]\!](\sigma) = \sigma'$ (in particular, $\mathcal{S}[\![\alpha]\!](\sigma)$ is defined).



$\langle q_\triangleright ; (3, 7) \rangle$

$\overset{\text{y := 1}}{\Longrightarrow}$ $\mathcal{S}[\![\text{y := 1}]\!](\sigma) = \sigma[\text{y} \mapsto 1]$

$\langle q_1 ; (3, 1) \rangle$

Execution Steps

We run program graphs by performing execution steps

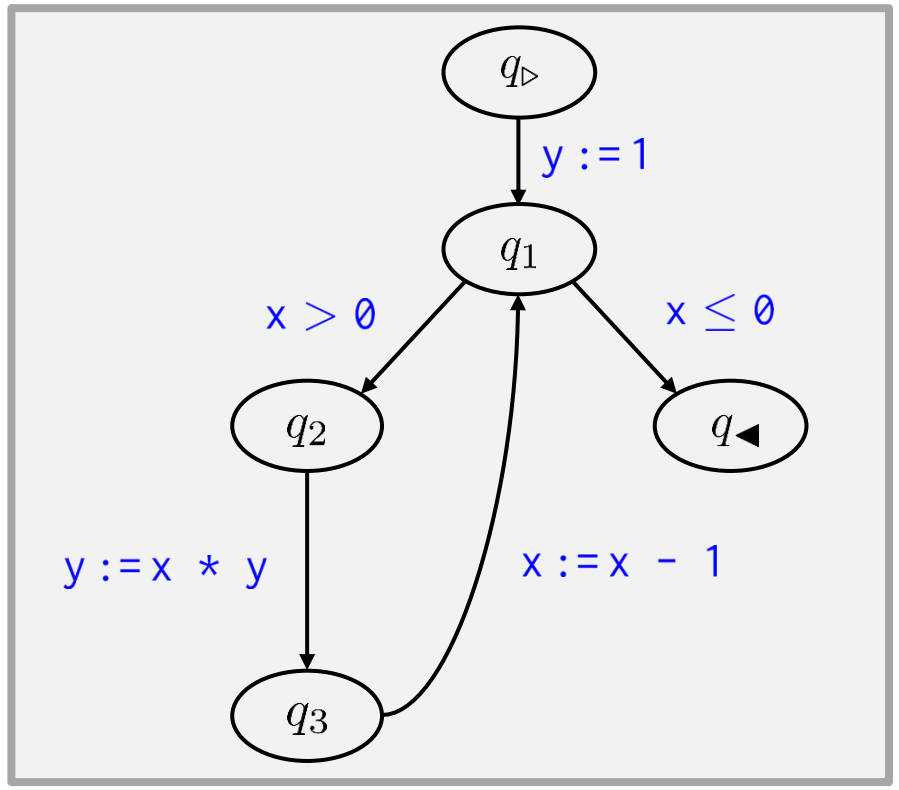
- Update program counter using edge relation **E**
- Update memory using semantic function $\mathcal{S}[\![\cdot]\!]$
- Updating memory undefined ➔ no execution step possible

Definition. We can perform an **execution step**

$$\langle q_\circ ; \sigma \rangle \overset{\alpha}{\Longrightarrow} \langle q_\bullet ; \sigma' \rangle$$

whenever

- $(q_\circ, \alpha, q_\bullet) \in$ **E** and
- $\mathcal{S}[\![\alpha]\!](\sigma) = \sigma'$ (in particular, $\mathcal{S}[\![\alpha]\!](\sigma)$ is defined).



$\langle q_1 ; (3, 1) \rangle$

$\overset{x \leq 0}{\Longrightarrow}$ $\mathcal{S}[\![x \leq 0]\!](\sigma) = \begin{cases} \sigma, & \text{if } \sigma(x) \leq 0 \\ \text{undefined}, & \text{if } \sigma(x) > 0 \end{cases}$

no step possible

Execution Steps

We run program graphs by performing execution steps

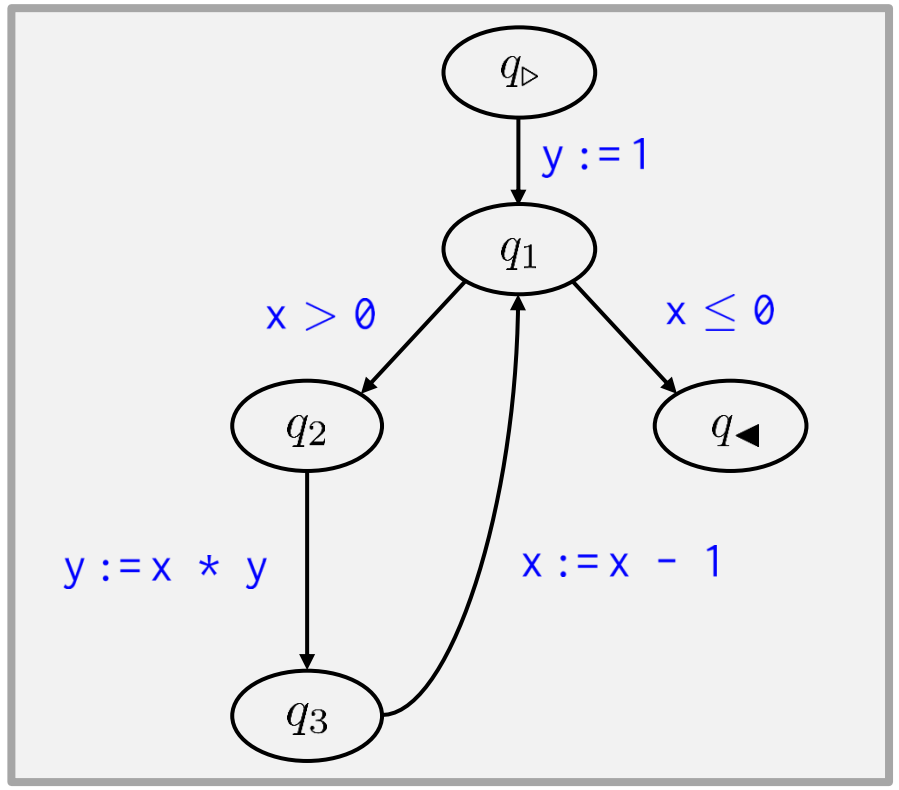
- Update program counter using edge relation **E**
- Update memory using semantic function $\mathcal{S}[\![\cdot]\!]$
- Updating memory undefined ➔ no execution step possible

Definition. We can perform an **execution step**

$$\langle q_\circ; \sigma \rangle \overset{\alpha}{\Longrightarrow} \langle q_\bullet; \sigma' \rangle$$

whenever

- $(q_\circ, \alpha, q_\bullet) \in$ **E** and
- $\mathcal{S}[\![\alpha]\!](\sigma) = \sigma'$ (in particular, $\mathcal{S}[\![\alpha]\!](\sigma)$ is defined).



$\langle q_1; (3, 1) \rangle$

$\overset{x>0}{\Longrightarrow}$ $\mathcal{S}[\![x > 0]\!](\sigma) = \begin{cases} \sigma, & \text{if } \sigma(x) > 0 \\ \text{undefined}, & \text{if } \sigma(x) \leq 0 \end{cases}$

$\langle q_2; (3, 1) \rangle$

Execution Sequences

Definition. An **execution sequence**

$$\langle q_\circ; \sigma \rangle \overset{\omega}{\Longrightarrow}{}^{*} \langle q_\bullet; \sigma' \rangle$$

of a (possibly empty) sequence of actions

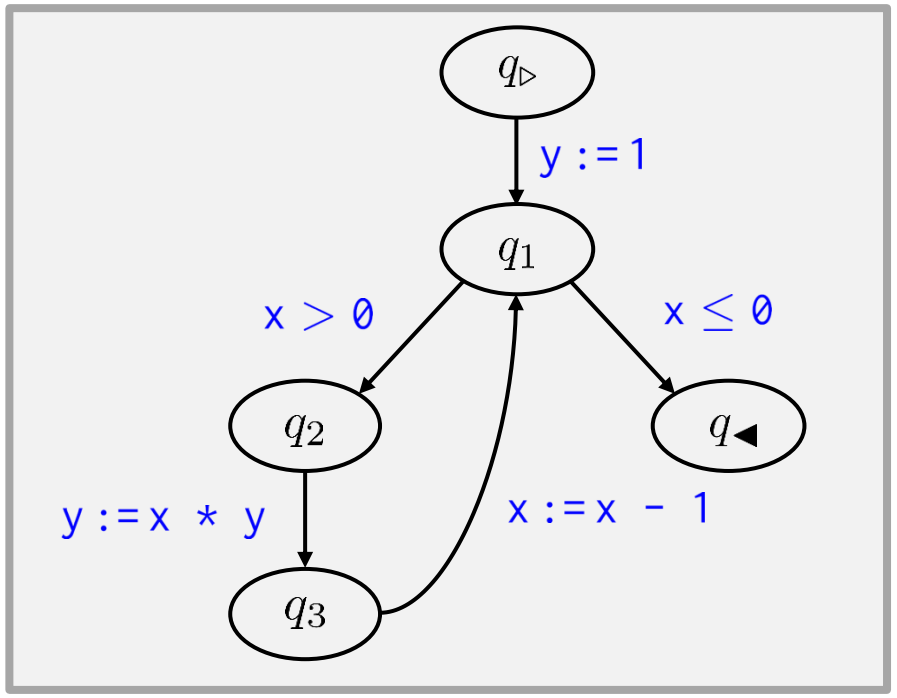
$$\omega = \alpha_1 \ldots \alpha_n$$

is a sequence of execution steps

$$\langle q_\circ; \sigma \rangle \overset{\alpha_1}{\Longrightarrow} \langle q_1; \sigma_1 \rangle \overset{\alpha_2}{\Longrightarrow} \ldots \overset{\alpha_{n-1}}{\Longrightarrow} \langle q_{n-1}; \sigma_{n-1} \rangle \overset{\alpha_n}{\Longrightarrow} \langle q_\bullet; \sigma' \rangle.$$

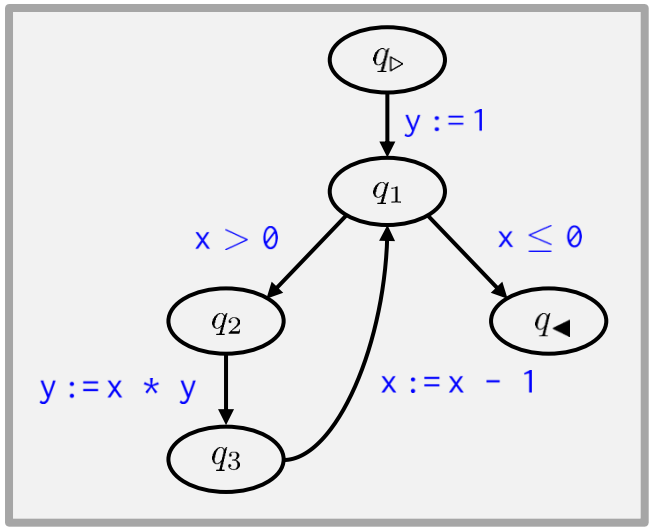
Definition. An execution sequence $\langle q_\circ; \sigma \rangle \overset{\omega}{\Longrightarrow}{}^{*} \langle q_\bullet; \sigma' \rangle$ is **complete** if $q_\circ = q_\triangleright$ and $q_\bullet = q_\blacktriangleleft$.

Complete execution sequence $\langle q_\triangleright; (3,7) \rangle \overset{\omega}{\Longrightarrow}{}^{*} \langle q_\blacktriangleleft; (0,6) \rangle$



$$\langle q_\triangleright; (3,7) \rangle \overset{\texttt{y := 1}}{\Longrightarrow} \langle q_1; (3,1) \rangle$$
$$\overset{\texttt{x>0}}{\Longrightarrow} \langle q_2; (3,1) \rangle \overset{\texttt{y := x*y}}{\Longrightarrow} \langle q_3; (3,3) \rangle$$
$$\overset{\texttt{x := x-1}}{\Longrightarrow} \langle q_1; (2,3) \rangle \overset{\texttt{x>0}}{\Longrightarrow} \langle q_2; (2,3) \rangle$$
$$\overset{\texttt{y := x*y}}{\Longrightarrow} \langle q_3; (2,6) \rangle \overset{\texttt{x := x-1}}{\Longrightarrow} \langle q_1; (1,6) \rangle$$
$$\overset{\texttt{x>0}}{\Longrightarrow} \langle q_2; (1,6) \rangle \overset{\texttt{y := x*y}}{\Longrightarrow} \langle q_3; (1,6) \rangle$$
$$\overset{\texttt{x := x-1}}{\Longrightarrow} \langle q_1; (0,6) \rangle \overset{\texttt{x≤0}}{\Longrightarrow} \langle q_\blacktriangleleft; (0,6) \rangle$$

Execution Sequences



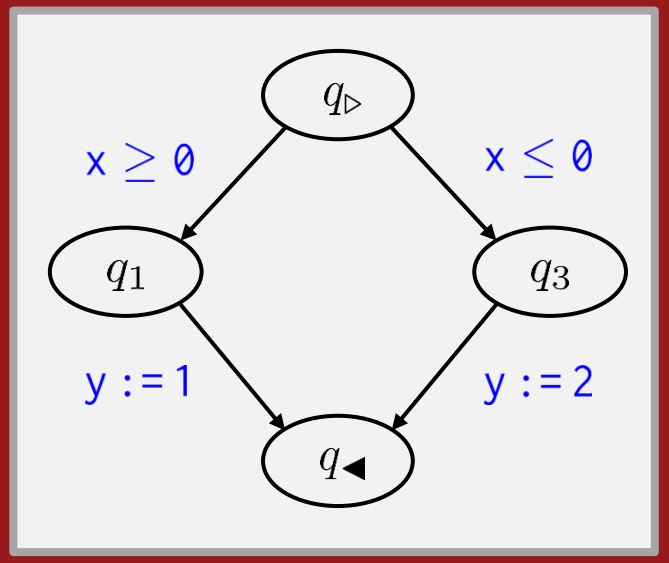
Definition. We write $\langle q_\circ; \sigma \rangle \Longrightarrow^* \langle q_\bullet; \sigma' \rangle$ to denote that $\langle q_\bullet; \sigma' \rangle$ is **reachable** from $\langle q_\circ; \sigma \rangle$ using zero or more (but finitely many) execution steps, i.e. there exists some ω such that $\langle q_\circ; \sigma \rangle \overset{\omega}{\Longrightarrow}^* \langle q_\bullet; \sigma' \rangle$.

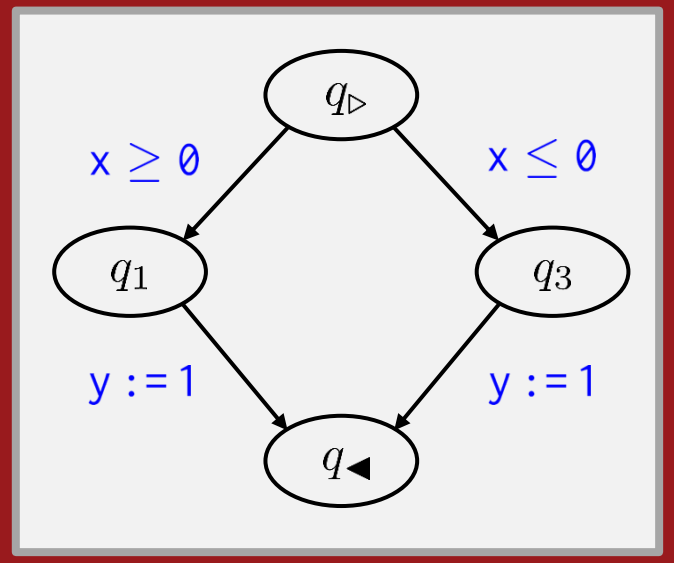
Which statements are true?

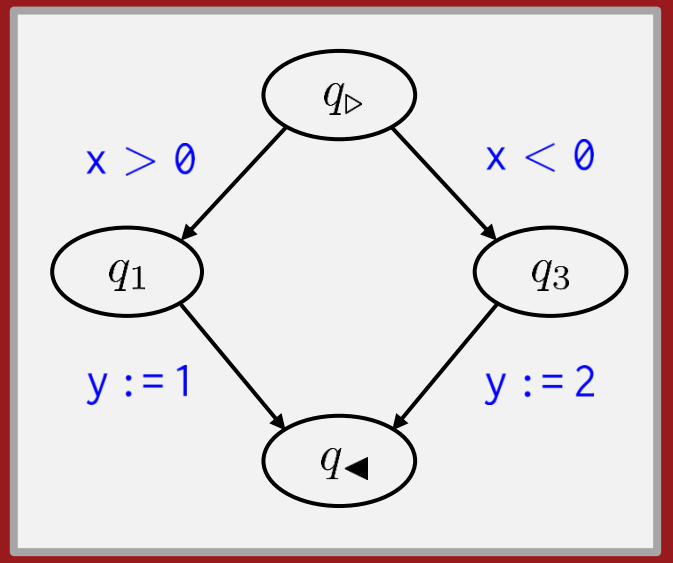
1. $\langle q_{\triangleright}; (-2, 7) \rangle \Longrightarrow^* \langle q_{\blacktriangleleft}; (-2, 7) \rangle$
2. $\langle q_2; (2, 4) \rangle \Longrightarrow^* \langle q_{\blacktriangleleft}; (0, 16) \rangle$
3. $\langle q_2; (2, 4) \rangle \Longrightarrow^* \langle q_1; (0, 8) \rangle$
4. $\langle q_{\triangleright}; (4, 17) \rangle \Longrightarrow^* \langle q_{\blacktriangleleft}; (0, 12) \rangle$

Exercise

- How many complete execution sequences (depending on the initial memory) do the following program graphs have?

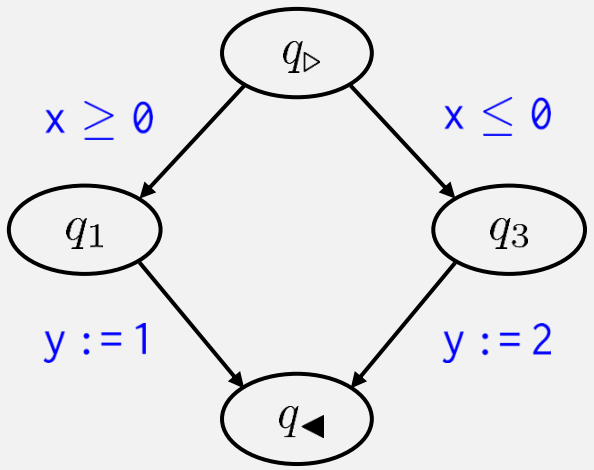






Solution

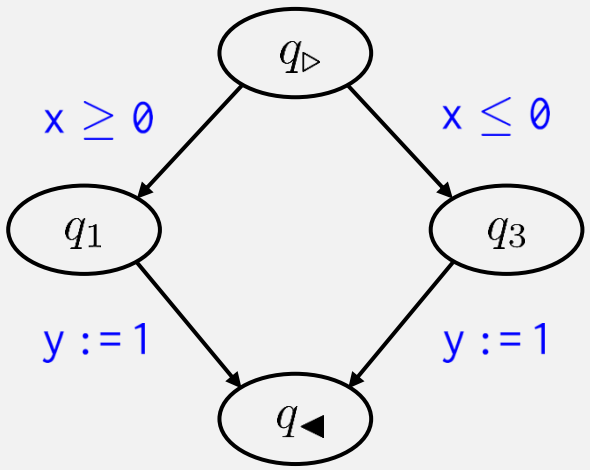
- How many complete execution sequences (depending on the initial memory) do the following program graphs have?



$\langle q_\triangleright; (0, m)\rangle \Longrightarrow^* \langle q_\blacktriangleleft; (0, 1)\rangle$

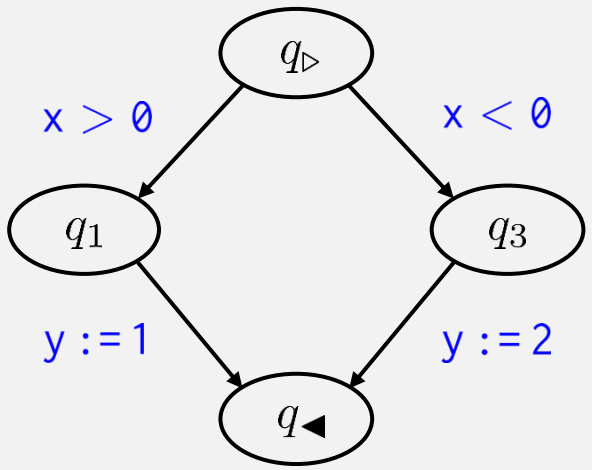
$\langle q_\triangleright; (0, m)\rangle \Longrightarrow^* \langle q_\blacktriangleleft; (0, 2)\rangle$

1 comp. seq. if $\sigma(\mathsf{x}) \neq 0$



$\langle q_\triangleright; (n, m)\rangle \Longrightarrow^* \langle q_\blacktriangleleft; (n, 1)\rangle$

2 comp. seq. if $\sigma(\mathsf{x}) = 0$



1 comp. seq. if $\sigma(\mathsf{x}) \neq 0$

no comp. seq. if $\sigma(\mathsf{x}) = 0$

(stuck in initial state)

Wrap-up: Operational Semantics

- **Goal:** formalize how programs are executed step by step
  - **Component 1:** control-flow modeled by a program graph
  - **Component 2:** semantic function modeling interaction with memory

$$\langle q_\circ; \sigma \rangle \overset{\omega}{\Longrightarrow}{}^* \langle q_\bullet; \sigma' \rangle$$

Definition. The **operational semantics** of a program graph and its semantic function is the *set of all complete execution sequences*.

Simple memory model:

$$\sigma : \textbf{Var} \to \textbf{Int}$$

Extensions: arrays, more types, lists, heaps, caches...

- Foundation for reasoning about programs

$$\forall n \geq 0 : \ \langle q_\triangleright; (n, m) \rangle \Longrightarrow^* \langle q_\blacktriangleleft; (0, n!) \rangle \ ?$$

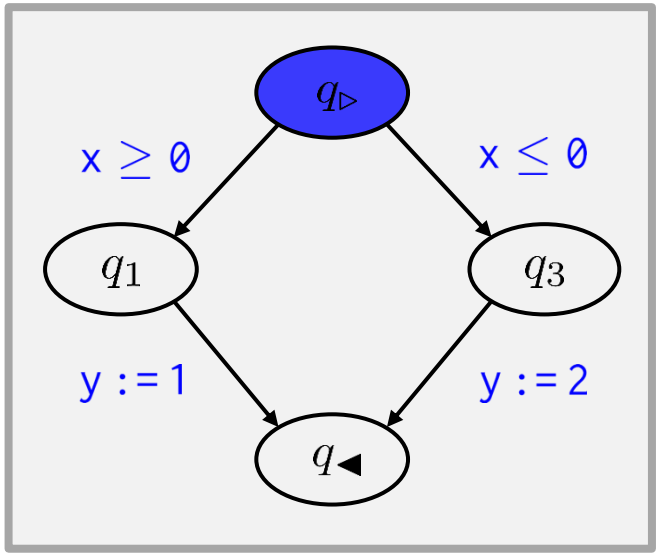
02141 – Computer Science Modelling

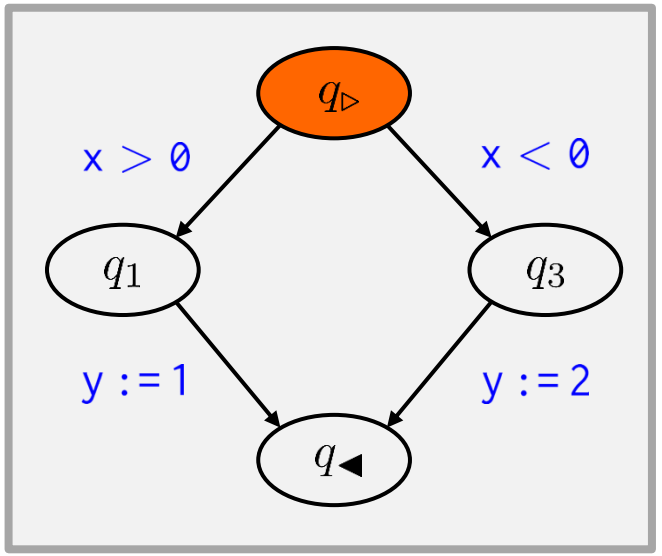
PROPERTIES OF PROGRAM GRAPHS

Properties of Systems

- **Observation:** configurations may admit *zero*, *one*, or *many* execution sequences
- Configurations may allow multiple executions sequences with different outputs for the same input
  - Examples: concurrency, randomness, caching effects
  - Undesirable: difficult to debug
- Configurations may become stuck: neither final nor admitting any further execution step
  - Examples: crashes, runtime errors, ill-defined programs
  - Undesirable: indicates faulty programs
- **Goal:** develop formal methods to ensure that such undesirable scenarios do not arise.

$\sigma(\mathsf{x}) = 0$

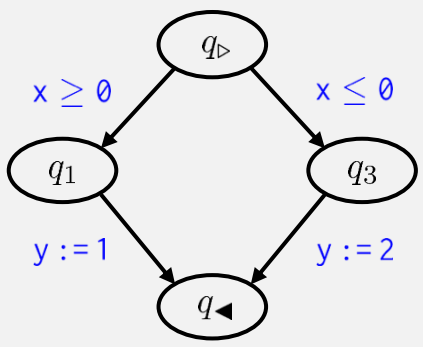




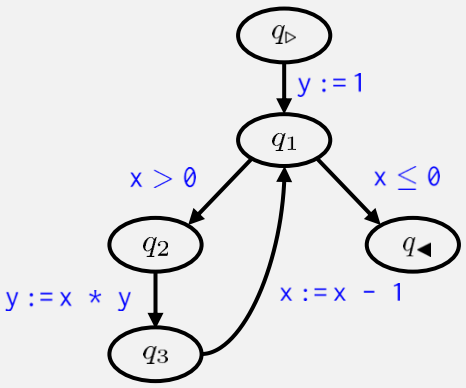
Deterministic Systems

- A deterministic system computes the same values by performing the same actions whenever it is started on the same input.

➔ rules out multiple executions for one input



non-deterministic

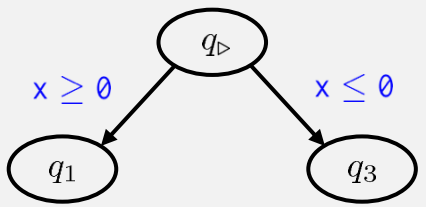


deterministic system

Definition. A program graph and its semantic function constitute a **deterministic system** if the following holds for all initial memories σ:

$$\text{if} \quad \langle q_\triangleright; \sigma\rangle \overset{\omega'}{\Longrightarrow}^* \langle q_\blacktriangleleft; \sigma'\rangle \ \text{ and } \ \langle q_\triangleright; \sigma\rangle \overset{\omega''}{\Longrightarrow}^* \langle q_\blacktriangleleft; \sigma''\rangle$$

then $\omega' = \omega''$ and $\sigma' = \sigma''$.



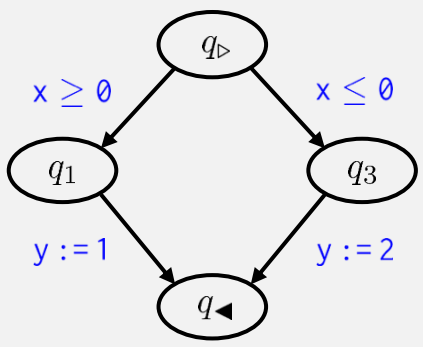
Issue:

$\mathcal{S}[\![\mathtt{x} \geq \mathtt{0}]\!](\sigma)$ and $\mathcal{S}[\![\mathtt{x} \leq \mathtt{0}]\!](\sigma)$ are defined simultaneously

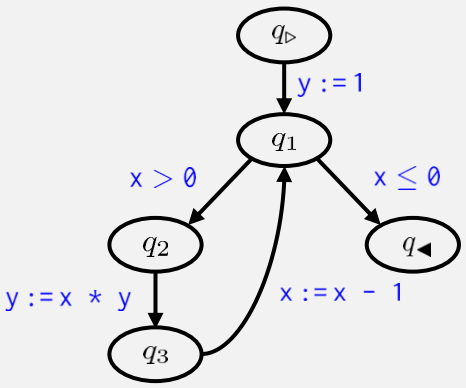
➔ prevent such cases

Deterministic Systems

Observation: a system is deterministic if distinct edges with the same source node are defined for disjoint sets of memories.



non-deterministic

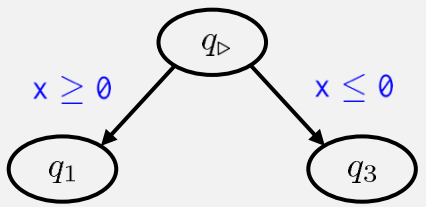


deterministic system

Theorem. A program graph **PG** and its semantic function constitute a **deterministic system** *if*

$$\forall (q, \alpha_1, q_1), (q, \alpha_2, q_2) \in \mathbf{E}\colon (\alpha_1, q_1) \neq (\alpha_2, q_2)$$

implies $\mathrm{dom}(\mathcal{S}[\![\alpha_1]\!]) \cap \mathrm{dom}(\mathcal{S}[\![\alpha_2]\!]) = \emptyset$.



Issue:

$\mathcal{S}[\![\mathtt{x} \geq \mathtt{0}]\!](\sigma)$ and $\mathcal{S}[\![\mathtt{x} \leq \mathtt{0}]\!](\sigma)$ are defined simultaneously

➔ prevent such cases

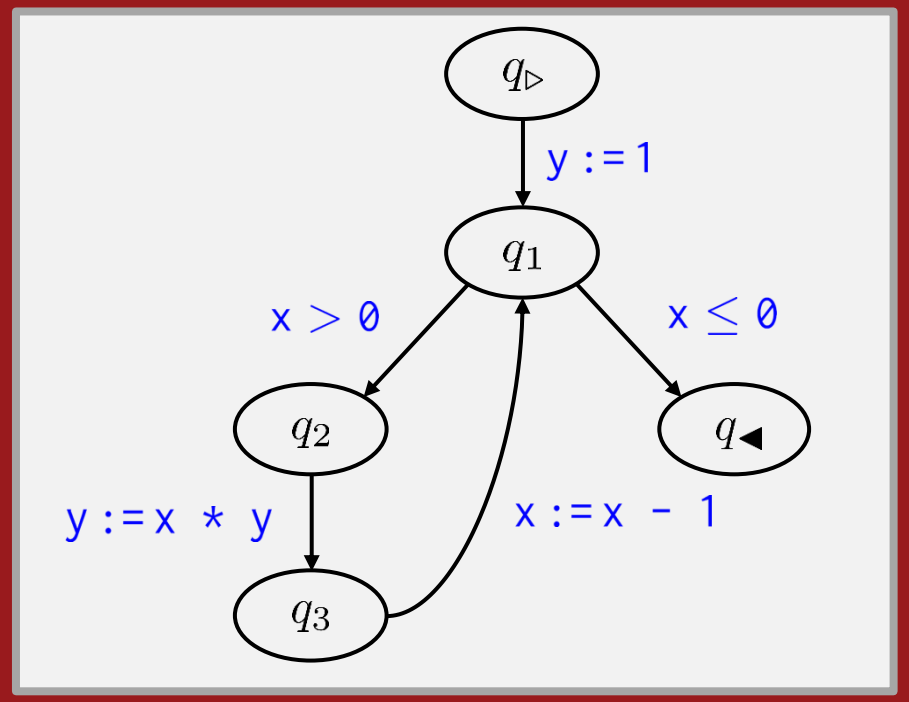
Exercise

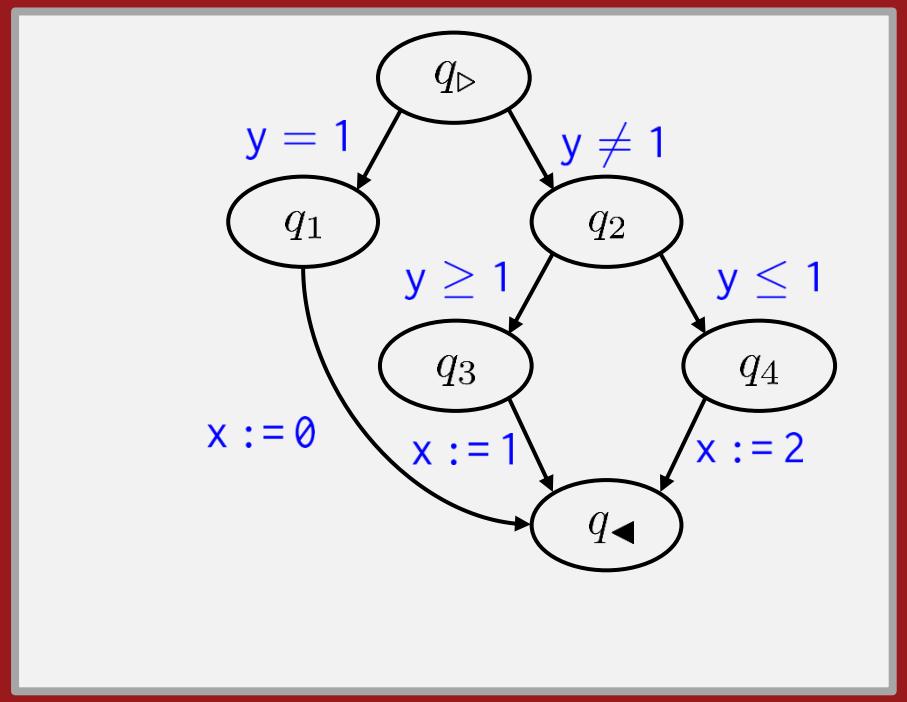
- Check which of the following systems are deterministic (for the canonical semantic function).

Theorem. A program graph **PG** and its semantic function constitute a **deterministic system** if

$$\forall (q, \alpha_1, q_1), (q, \alpha_2, q_2) \in \mathbf{E}\colon (\alpha_1, q_1) \neq (\alpha_2, q_2)$$

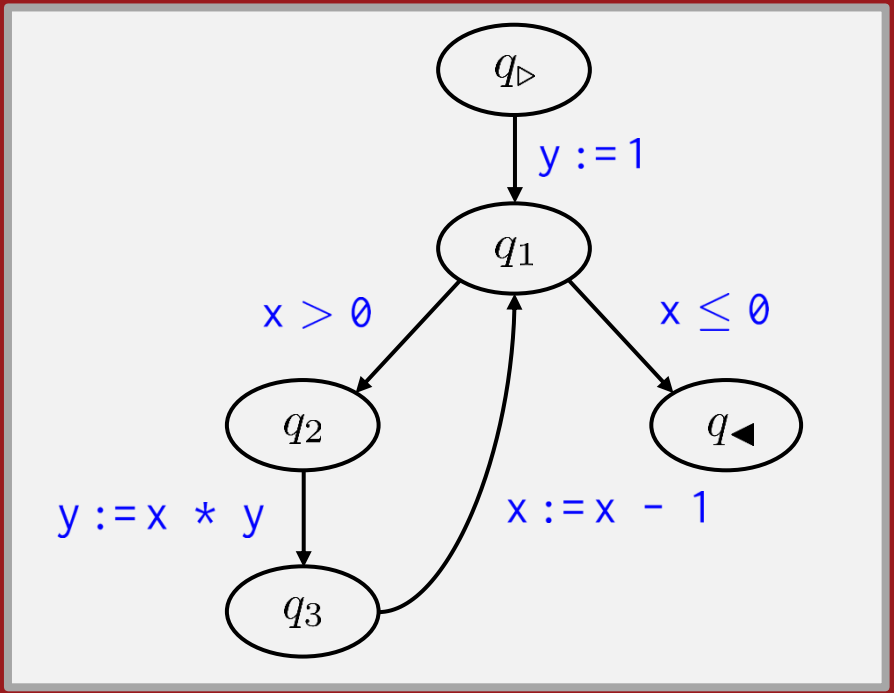
implies $\mathrm{dom}(\mathcal{S}[\![\alpha_1]\!]) \cap \mathrm{dom}(\mathcal{S}[\![\alpha_2]\!]) = \emptyset$.





Solution

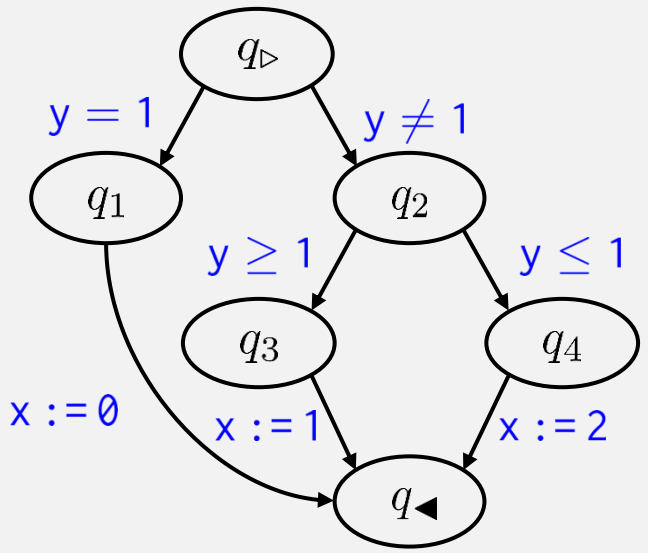
- Check which of the following systems are deterministic (for the canonical semantic function).



Theorem. A program graph **PG** and its semantic function constitute a **deterministic system** if

$$\forall (q, \alpha_1, q_1), (q, \alpha_2, q_2) \in \mathbf{E}\colon (\alpha_1, q_1) \neq (\alpha_2, q_2)$$

implies $\mathrm{dom}(\mathcal{S}[\![\alpha_1]\!]) \cap \mathrm{dom}(\mathcal{S}[\![\alpha_2]\!]) = \emptyset$.

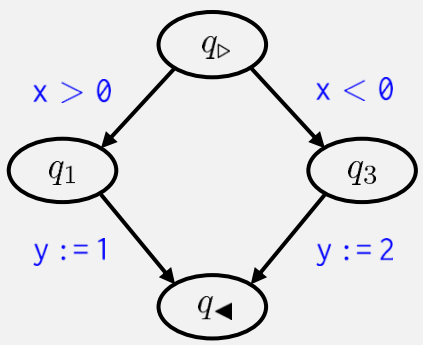


deterministic system, despite overlapping semantic functions

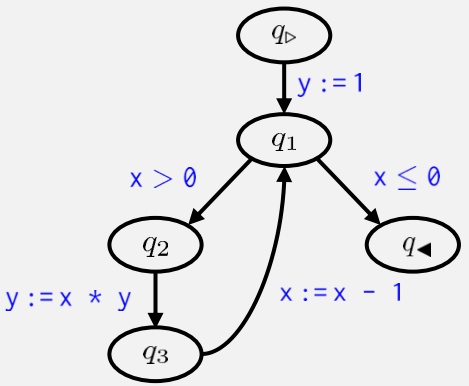
The condition in the above theorem is **sufficient** but **not necessary** for determinism.

Evolving Systems

- An evolving system can always perform execution steps unless it reached a final configuration.



non-evolving



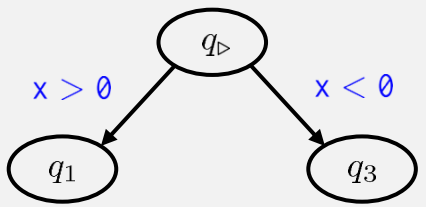
evolving system

Definition. A program graph and its semantic function constitute an **evolving system** whenever

$$\langle q_\triangleright; \sigma\rangle \overset{\omega}{\Longrightarrow}^* \langle q'; \sigma'\rangle \text{ with } q' \neq q_\blacktriangleleft$$

can always be extended by some $\alpha \in$ **Act** to

$$\langle q_\triangleright; \sigma\rangle \overset{\omega}{\Longrightarrow}^* \langle q'; \sigma'\rangle \overset{\alpha}{\Longrightarrow}^* \langle q''; \sigma''\rangle.$$



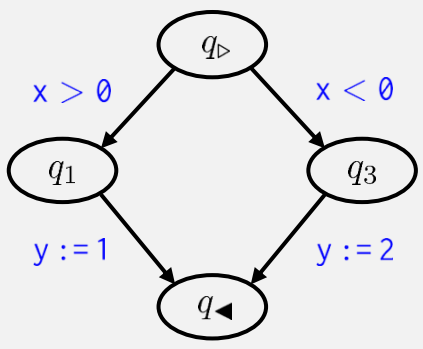
Issue:

$\mathcal{S}[\![\mathtt{x} > \mathtt{0}]\!](\sigma)$ and $\mathcal{S}[\![\mathtt{x} < \mathtt{0}]\!](\sigma)$ can *both* be *un*defined

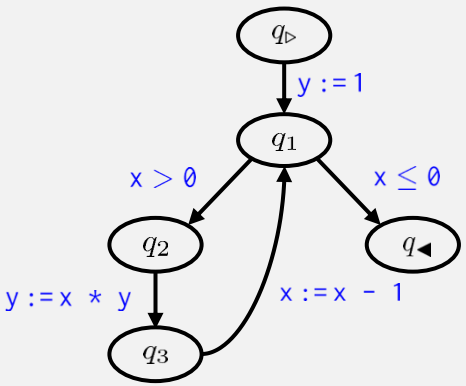
➔ prevent such cases

Evolving Systems

Observation: a system is evolving if, for every node and every memory σ, there is an edge from that node whose semantic function is defined for σ.



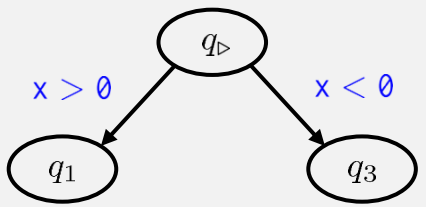
non-evolving



evolving system

Theorem. A program graph **PG** and its semantic function constitute an **evolving system** if

$$\forall q \in \mathbf{Q} \setminus \{q_\blacktriangleleft\}\ \forall \sigma \in \mathbf{Mem}$$
$$\exists (q, \alpha, q') \in \mathbf{E}: \quad \sigma \in \mathrm{dom}(\mathcal{S}[\![\alpha]\!]).$$



Issue:

$\mathcal{S}[\![\mathtt{x} > \mathtt{0}]\!](\sigma)$ and $\mathcal{S}[\![\mathtt{x} < \mathtt{0}]\!](\sigma)$ can *both* be *un*defined

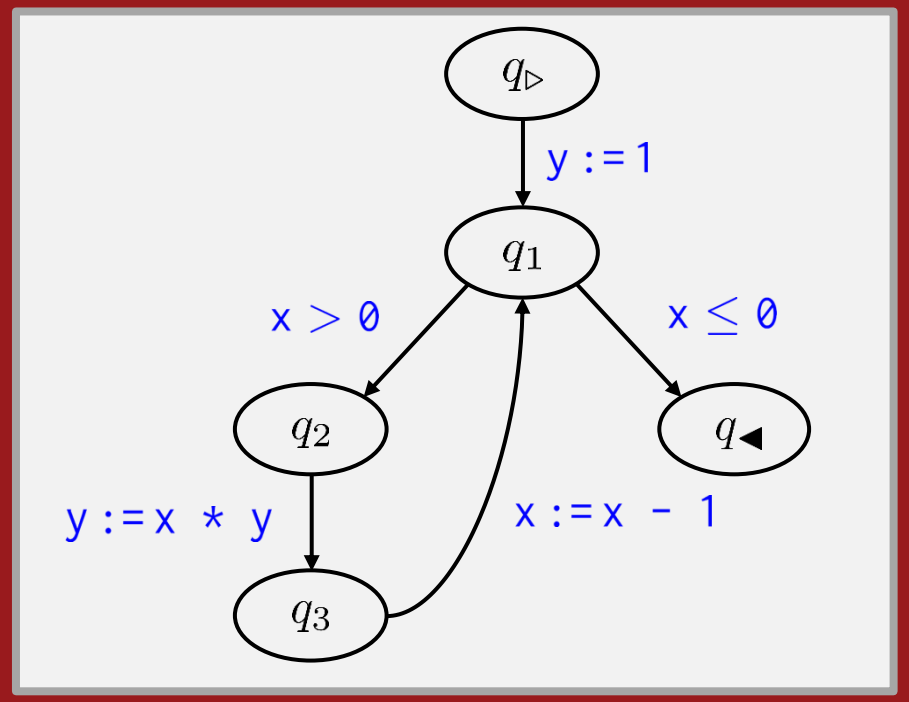
➔ prevent such cases

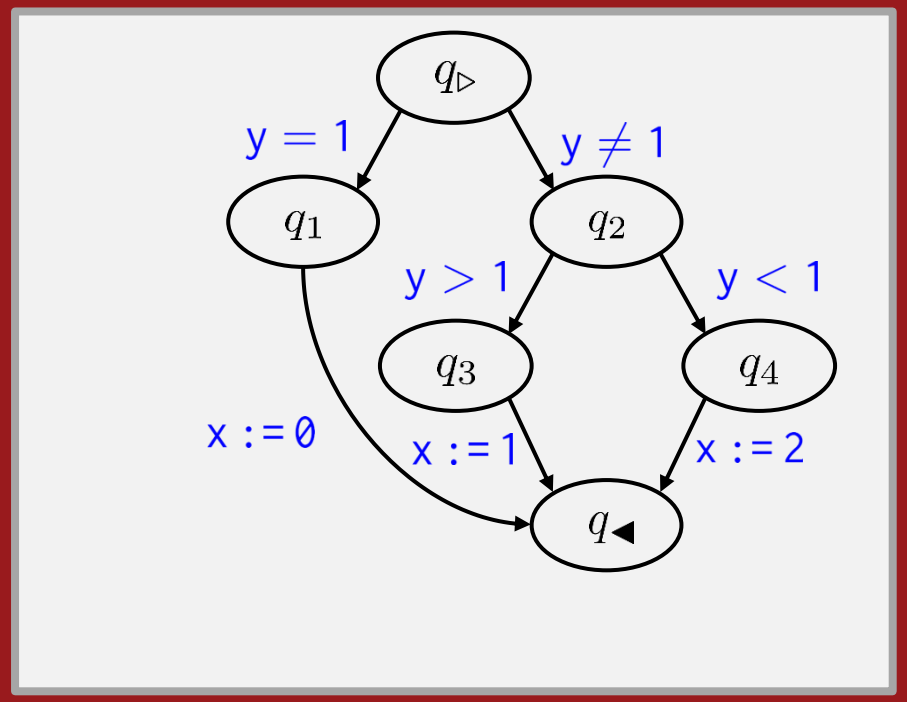
Exercise

- Check which of the following systems are evolving (for the canonical semantic function).

Theorem. A program graph **PG** and its semantic function constitute an **evolving system** if

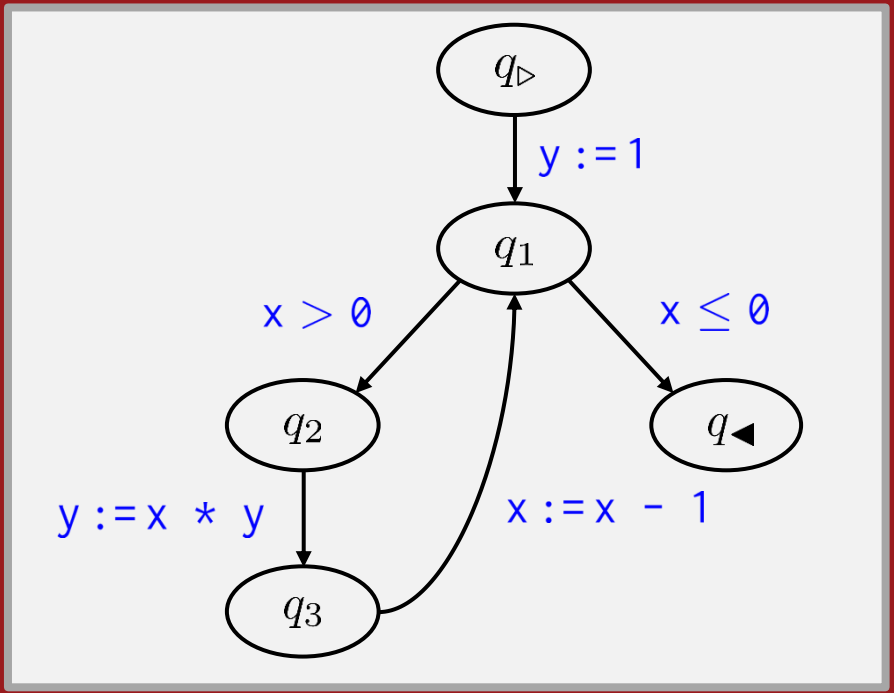
$$\forall q \in \mathbf{Q} \setminus \{q_{\blacktriangleleft}\}\ \forall \sigma \in \mathbf{Mem}$$
$$\exists (q, \alpha, q') \in \mathbf{E}: \quad \sigma \in \mathrm{dom}(\mathcal{S}[\![\alpha]\!]).$$





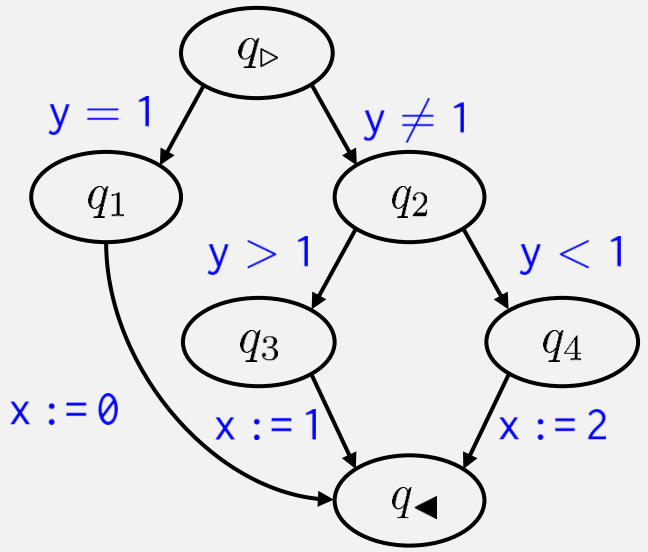
Solution

- Check which of the following systems are evolving (for the canonical semantic function).



Theorem. A program graph **PG** and its semantic function constitute an **evolving system** if

$$\forall q \in \mathbf{Q} \setminus \{q_\blacktriangleleft\}\ \forall \sigma \in \mathbf{Mem}$$
$$\exists (q, \alpha, q') \in \mathbf{E}: \quad \sigma \in \mathrm{dom}(\mathcal{S}[\![\alpha]\!]).$$



evolving system, despite semantic function not defined in edges from q2

The condition in the above theorem is **sufficient** but **not necessary** for having an evolving system.

Wrap-Up: Properties of Program Graphs

- A deterministic system computes the same values by performing the same actions whenever it is started on the same input
  - ➔ rules out multiple executions for one input
  - ➔ avoids complicated debugging and testing

- An evolving system can always perform execution steps unless it reached a final configuration
  - ➔ rules out getting stuck
  - ➔ avoids runtime errors, memory issues, etc.

- Sufficient but not necessary criteria for checking the above two properties

Formal Methods Workflow

identify properties of interest

⇩

formalize properties in terms of (operational) semantics

⇩

find methods for *efficiently checking* the properties

What next?

- Formalizing a programming language (GCL)
- Control flow via program graph
- Semantic functions for statements
- More intricate memory models

Exercise Class

- Today's exercises: DTU Learn > Content > FM1 > FM1-exercises.pdf
  - Ask us for feedback and/or help!

Thanks for today!